



Digital VLSI System Design

Fall 2024

HW2 – implementation of CORDIC algorithm

EE Department, Sharif University of Technology

Dr. Shabany

Deadline: 13 Aban

Instructions for Completing this Assignment:

1. To submit your assignment, upload all files, including modules, testbenches, MATLAB files and a report detailing the process along with relevant results to the designated system. For code submission, organize the files related to modules and testbenches in separate folders (each section should also be placed in individual folders). For sections that include additional files like memory or text files, make sure to submit those as well. Avoid submitting the entire project!
2. The code you submit must be synthesizable and simulatable. If the code cannot be synthesized or simulated, no points will be awarded.
3. Collaboration, seeking help from peers, and searching online or in books is allowed, but the assignment must be completed by you individually. Any unusual similarities will result in zero points for all involved parties.
4. In your coding tasks, try to use both behavioral and dataflow modeling so that you become proficient in both levels of abstraction.
5. If you have any questions or concerns regarding the assignment, raise them in the course's Telegram group.
6. You must provide a report describing the work done, along with a block diagram of the implemented structure, the required explanations, and simulation results that verify its correctness. Use MATLAB to validate the accuracy of all questions.

In this assignment, you will become familiar with the CORDIC algorithm, used to compute certain trigonometric, hyperbolic, and logarithmic functions. CORDIC is an efficient and high-speed method that can be leveraged for hardware computation of such functions. One key advantage of this algorithm is that it allows the implementation of trigonometric functions without using a multiplier. In this assignment, you will focus on implementing the **atan** function using CORDIC and then verify and test your implementation. Finally, you will provide a report that includes performance metrics such as speed, area, and power consumption.

[Article 1](#) details a circuit optimization method for calculating **atan**, including implementation specifics. Therefore, it is necessary to thoroughly and carefully study this paper before proceeding.

Work Process:

1. First, implement the algorithm for calculating **atan** in MATLAB, based on the referenced paper. When implementing an algorithm in MATLAB for eventual hardware design, ensure that the final MATLAB code is as aligned as possible with the design that will be implemented. Hence, while designing the MATLAB code, consider the constraints and limitations of hardware implementation.
2. Next, generate random Test Vectors within a valid range using MATLAB, and apply these values to your MATLAB code to obtain the output. The generated Test Vectors should cover the entire valid range.
3. To validate your MATLAB code, compare the results of the **atan** function computed by your MATLAB code against the results from MATLAB's built-in **atan** function. Plot both sets of results to confirm the correctness of your MATLAB code.
4. After verifying your MATLAB implementation, implement the same algorithm using a Hardware Description Language (HDL), specifically Verilog.
5. Simulate the implemented design. At this stage, you should write the Test Vectors from step 2 to a text file, considering the input constraints of the design implemented in step 4. These values should then be read from the text file into the hardware module, and the

module's output should also be written to a text file. Compare the results by reading the output text file in MATLAB and plotting it against the results from step 2 to verify the accuracy of the hardware implementation. (For simulation, you can use both Modelsim or Vivado softwares.)

6. Once the correctness of your circuit is verified, synthesize and implement the design using Vivado (on Zynq7010 chip – xc7z010clg400-1). Finally, based on the given constraints, particularly regarding clock frequency, provide a report detailing the area, speed, and power consumption of your design. Through experimentation, determine the maximum clock frequency (i.e., the highest speed) that your circuit can achieve. Start by setting the clock frequency to 50 MHz, then determine the highest possible clock frequency.

Important Notes and Constraints:

1. For this assignment, a comprehensive report must be submitted detailing the entire process, along with the required documentation. The report should include the design process, algorithm details, implementation challenges, and results. For each section, provide visual outputs of the algorithm's results in both MATLAB and HDL. Additionally, the report should include the area of your design (i.e., the number of resources used), power consumption, and the highest clock frequency achieved.
2. Based on the referenced article, treat **W** (the number of fractional bits in the output) and the number of **iterations** as parameters in your module. Report the results for different values of these parameters.
3. Use unrolled pipelined architecture for this CORDIC processor according to the referenced article.
4. Make sure to comment on your code and use appropriate names for modules and variables to enhance code readability.

Implementation on Zynq:

In the next phase, you need to implement your CORDIC module on the FPGA of a Zynq chip. Using the PS (Processing System) and any communication method between PS and PL (Programmable Logic) learned from [Zynq videos](#), generate 100,000 random input data in the PS. Then, feed this data in a correctly synchronized manner with the clock to the module, and verify the accuracy of the output in the PS. Report the accuracy rate.

(To correctly complete this part, the clock generated by the PS must be fed into the CORDIC module, and the random input data must be sent to the PL synchronized with this clock. The outputs should also be returned to the PS in a specific order, where they can be compared with the correct **atan** values computed by the PS functions, and the accuracy percentage should be reported.)

Bonus Task: In addition to your own CORDIC module, use Vivado's CORDIC IP Core and compare the results of both in the PS, reporting the accuracy of each.

To complete this section, make sure to watch [the Zynq videos](#) as per [the provided schedule](#). The timing for implementation and testing on the board will be announced later, and if necessary, additional details on implementing on Zynq will be provided.

Good luck!